❑ Programming exercise 1

Build a small-scale system using the Cranfield dataset or a self-collected text corpus:

1. Represent documents using TF-IDF.
2. Input a query and return the top 10 documents.
3. Allow users to mark documents as relevant or non-relevant.
4. Update the query using the Rocchio algorithm.
5. Display the added terms along with their weights.
6. Re-run the retrieval and compare the two ranked lists.
7. Evaluate Precision@10, MAP, or NDCG before and after feedback.
8. Experiment with various values for α, β, γ.
9. Compare explicit feedback with pseudo-relevance feedback.
10. A small-scale experiment could utilize

$$\alpha \in \{0.5, 1, 1.5\}, \beta \in \{0.25, 0.5, 0.75, 1\}, \gamma \in \{0, 0.1, 0.25, 0.5\}$$

Analyze not only which parameter set yields the highest MAP but also when query drift occurs and which terms trigger this phenomenon.

❑ Programming exercise 2

❖Build and evaluate a PRF system:

- Use BM25 as the baseline
- Select Top-k feedback documents
- Implement Rocchio and RM3
- Select Top-m expansion terms
- Perform a second retrieval
- Evaluate MAP and NDCG@10
- Analyze query drift
- Compare different k, m, and feedback weights

❖Suggested datasets:

- Cranfield
- CISI
- TREC collections
- MS MARCO subsets
- BEIR datasets

❑ Programming exercise 3

Construct a small statistical thesaurus:

1. Select a document collection
2. Tokenize and remove stop words
3. Use a context window of size 5
4. Build co-occurrence counts
5. Calculate PPMI
6. Find Top-10 neighbors for each query term
7. Expand queries with Top-m terms
8. Retrieve using BM25
9. Compare MAP and NDCG@10

❑ Programming exercise 4

Build a text-classification system on:

- 20 Newsgroups
- Reuters-21578
- AG News
- BBC News
- A Vietnamese news dataset

Tasks:

1. Create a leakage-free pipeline
2. Implement Naive Bayes, Rocchio, and kNN
3. Tune hyperparameters using validation data
4. Report macro-F1 and micro-F1
5. Compare runtime and memory
6. Analyze the confusion matrix
7. Explain five misclassified documents
8. Identify the best model under different deployment constraints

❑ Programming exercise 5

Build a two-stage ranking system:

1. Use BM25 to retrieve Top-100 documents
2. Construct query–document features
3. Train a pointwise regression model
4. Train a Ranking SVM
5. Rerank the same candidate sets
6. Evaluate MAP and NDCG@10
7. Compare pairwise accuracy
8. Analyze feature weights
9. Analyze queries improved or harmed by LTR

❖Suggested datasets:

- LETOR 4.0
- MSLR-WEB10K
- MSLR-WEB30K
- Yahoo! Learning to Rank
- TREC collections

❑ Programming exercise 6

Build a small web crawler using Python with the following requirements:

1. Accept a seed URL.
2. Crawl only pages within the same domain.
3. Traverse using breadth-first search.
4. Respect robots.txt.
5. Normalize URLs.
6. Do not re-download previously visited URLs.
7. Limit the crawl to a maximum of 100 pages.
8. Wait at least one second between requests.
9. Store the URL, title, content, and out-links.
10. Visualize the link graph using NetworkX.
11. Report the number of nodes, number of edges, in-degree, out-degree, and strongly connected components.

❑ Programming exercise 7

❖Design a focused crawler for academic AI publications.

❖Determine:

1. Seed URLs
2. Relevance scoring method
3. Frontier policy
4. Politeness policy
5. Duplicate detection
6. Re-crawling strategy
7. Evaluation metrics

❑ Programming exercise 8

Develop a Python program with the following requirements:

1. Read a directed graph from an edge list.
2. Implement PageRank using power iteration (do not call `networkx.pagerank` directly).
3. Handle dangling nodes.
4. Allow the damping factor to be adjusted.
5. Terminate when the L_1 error falls below 10^{-6}.
6. Implement HITS with L_2 normalization.
7. Compare results with NetworkX.
8. Visualize the graph, with node sizes proportional to PageRank values.
9. Use distinct colors to represent hubs and authorities.
10. Analyze the impact of adding a small link farm to the graph.

❑ Programming exercise 9 - Anchor text extraction

❖Write a program that takes an HTML page and outputs the fields:

source_url

target_url

normalized_target_url

anchor_text

anchor_context

is_internal

is_image_link

❖Requirements:

- Handle relative URLs.
- Decode HTML entities.
- Merge nested text nodes.
- Use alt text for image links.
- Remove fragments from URLs.
- Normalize whitespace.

❑ Programming exercise 10 - Building an Anchor Index

Starting with a small set of web pages:

- ❖Collect all links.
- ❖Group anchor texts by target URL.
- ❖Remove duplicate anchors from the same source page.
- ❖Calculate term frequency within the anchor field.
- ❖Count the number of unique source domains.
- ❖Build an inverted index for anchor text.
- ❖Return the top 10 pages relevant to the query.

❑ Programming exercise 11 - Comparison of ranking models

❖Experimental configurations:

$$S_1(d,q) = BM25_{\text{body}}(d,q)$$

$$S_2(d,q) = BM25_{\text{anchor}}(d,q)$$

$$S_3(d,q) = \alpha BM25_{\text{body}}(d,q) + \beta BM25_{\text{anchor}}(d,q) + \gamma PageRank(d)$$

❖Evaluation metrics: Precision@10, MRR, MAP, NDCG@10

❖Subsequent analysis of effectiveness for:

- Navigational queries
- Informational queries
- HTML pages
- PDF documents and non-textual resources.

❑ Programming exercise 12 - LSI

❖Build a TF-IDF matrix for a small collection (e.g., Cranfield or Vietnamese news); compute a truncated SVD (k = 50, 100, 200); compare MAP with BM25.

❑ Programming exercise 13 - Dense retrieval

❖Encode passages with a pretrained model (e.g., multilingual-E5 or BGE-M3); index with FAISS (flat vs. HNSW); measure Recall@100 and latency

❑ Programming exercise 14 - Hybrid + rerank

❖Combine BM25 and dense results with RRF; rerank the top 100 with a cross-encoder; report NDCG@10 and MRR@10 with significance tests.